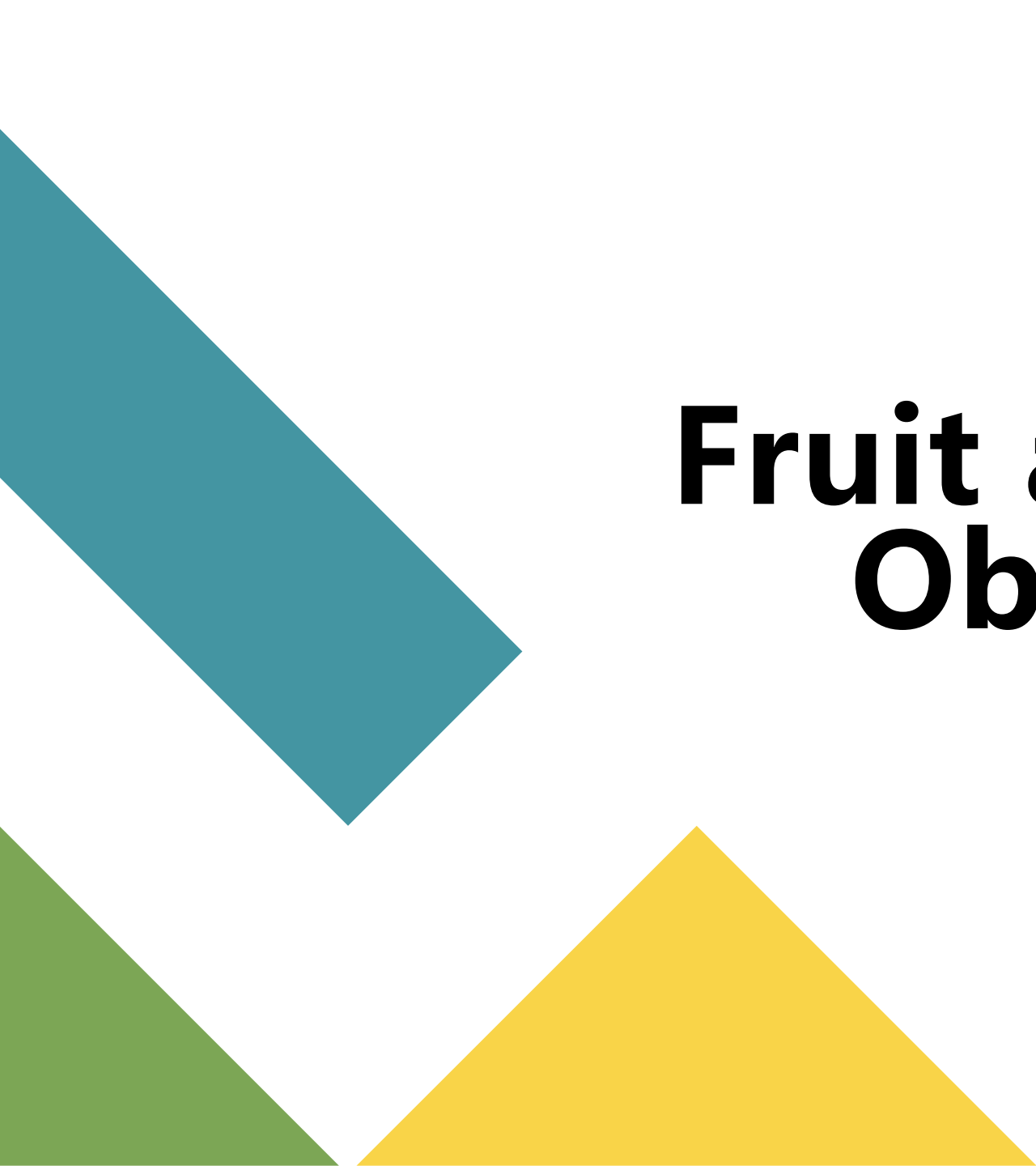




Fruit and Vegetable Object Detection

Content

- Introduction
- System setup
- Stream lit application



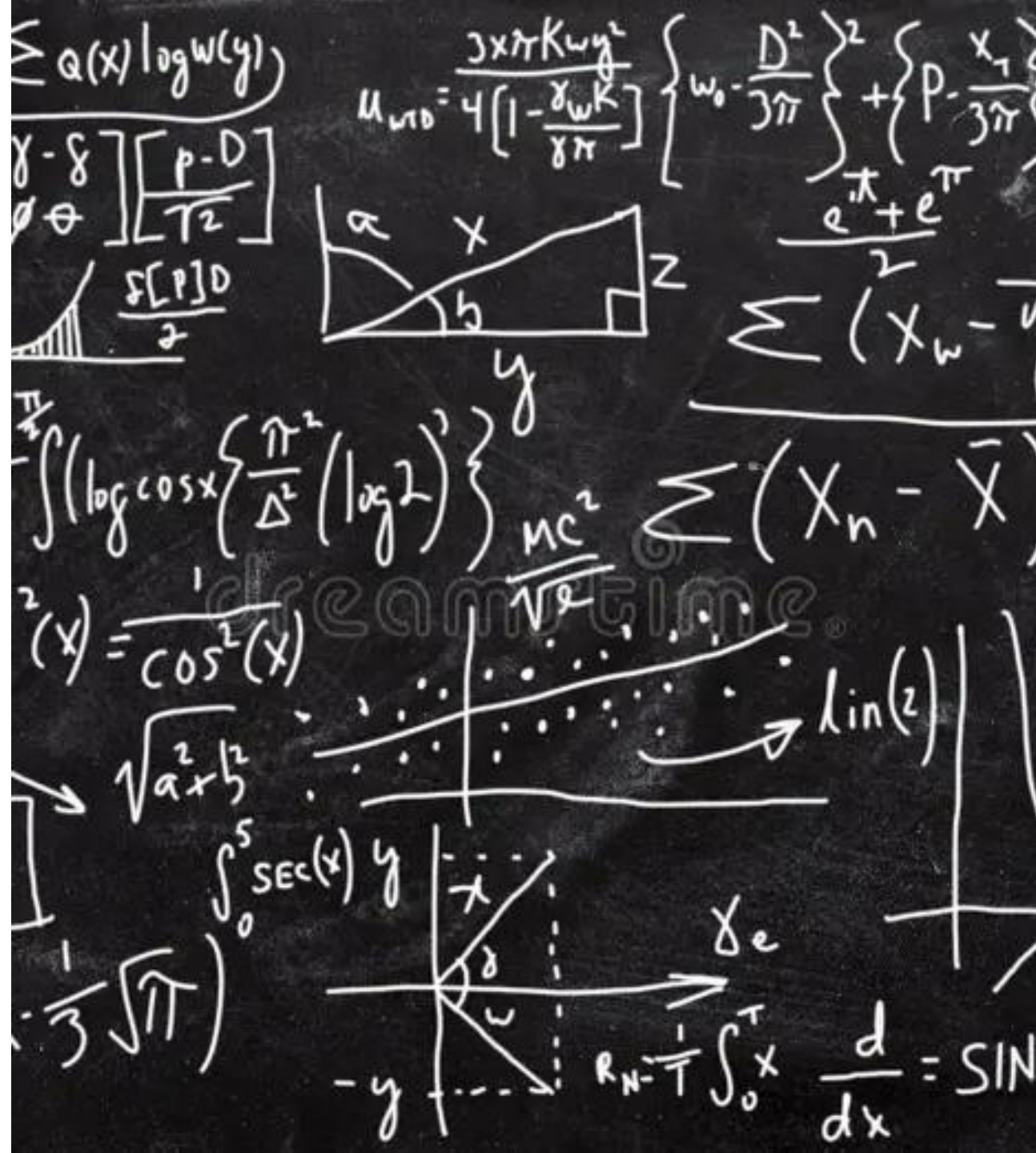


Introduction

Function

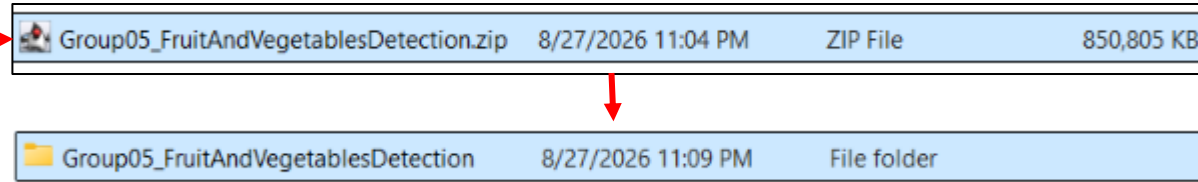
- This system allow users know the objects by detect the fruits and vegetables on the image and picture by the trained model
- Users can view the results after image or picture is processed
- The models performance also will be shown for users to compare
- The result cannot guarantee the fruits and vegetables is one hundred percent that is exactly what is it, but almost possible what it is

System Setup

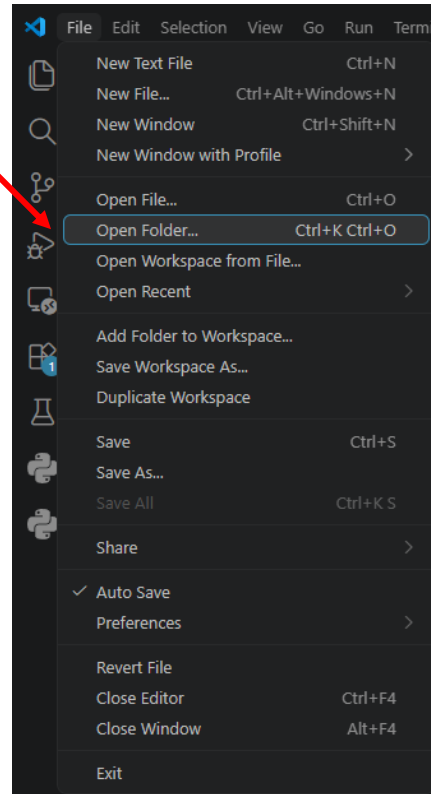


Step 1

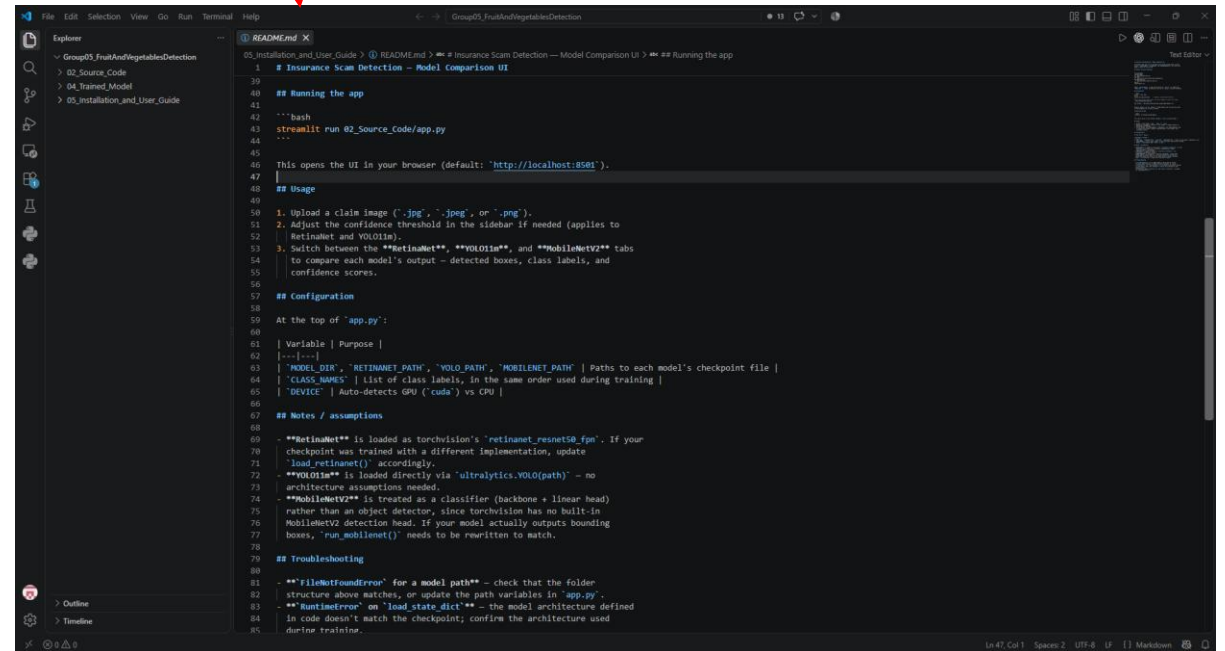
1. Unzip the folder



2. Open VS code and open unzipped folder



3. Open readme file



Step 2

1. Open Terminal

2. Enter “python -m venv venv”

A venv folder will appear after command entered

3. Enter “.\venv\Scripts\Activate.ps1”

If the scripts is disabled please enter “Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass” to unlock the folder then enter “.\venv\Scripts\Activate.ps1”

The final result is you will enter a venv state

The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane displays the project structure: Group05_FruitAndVegetablesDetection, 02_Source_Code, 04_Trained_Model, 05_Installation_and_User_Guide, and venv. The main editor shows the README.md file with installation instructions. The Terminal pane at the bottom shows the execution of the commands to create and activate the virtual environment.

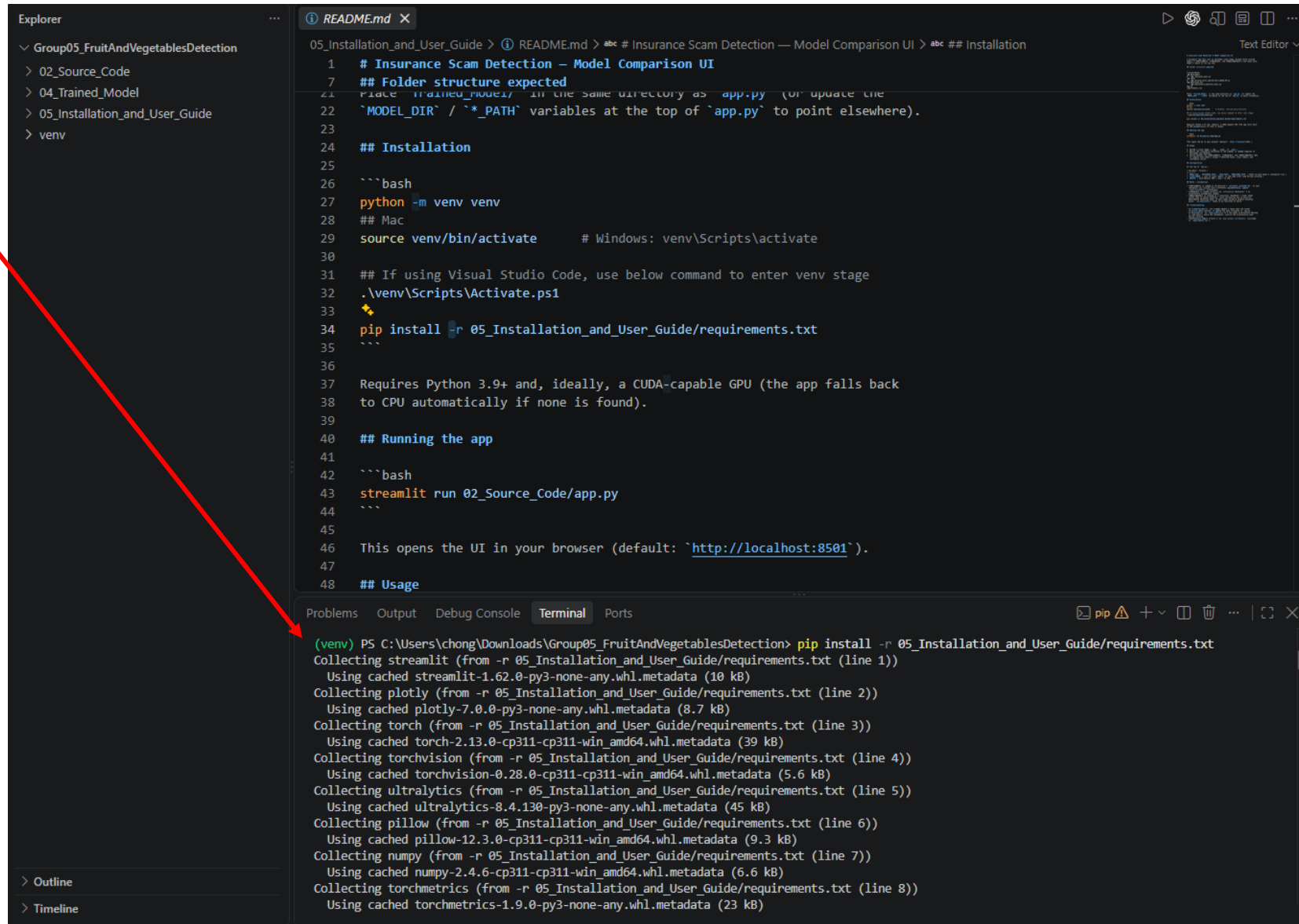
```
05_Installation_and_User_Guide > README.md > # Insurance Scam Detection — Model Comparison UI > ## Installation
1  # Insurance Scam Detection — Model Comparison UI
7  ## Folder structure expected
8
9  ...
10 Trained_Model/
11   |__ RetinaNet/
12      |__ retinanet_best.pt
13      |__ VOLO/
14         |__ yolo11m_focal_epoch4_best_map50_95.pt
15         |__ Mobilenet_V2/
16            |__ mobilenetv2_detector_best.pt
17 app.py
18 requirements.txt
19 ...
20
21 Place 'Trained_Model/' in the same directory as 'app.py' (or update the
22 'MODEL_DIR' / '*_PATH' variables at the top of 'app.py' to point elsewhere).
23
24 ## Installation
25
26 ```bash
27 python -m venv venv
28 ## Mac
29 source venv/bin/activate      # Windows: venv\Scripts\activate
30
31 ## If using Visual Studio Code, use below command to enter venv stage
32 .\venv\Scripts\Activate.ps1
33
34 pip install -r 05_Installation_and_User_Guide/requirements.txt
35 ...
```

```
PS C:\Users\chong\Downloads\Group05_FruitAndVegetablesDetection> python -m venv venv
PS C:\Users\chong\Downloads\Group05_FruitAndVegetablesDetection> .\venv\Scripts\Activate.ps1
.\venv\Scripts\Activate.ps1 : File C:\Users\chong\Downloads\Group05_FruitAndVegetablesDetection\venv\Scripts\Activate.ps1 cannot
be loaded because running scripts is disabled on this system. For more
information, see about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ .\venv\Scripts\Activate.ps1
+ ~~~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\chong\Downloads\Group05_FruitAndVegetablesDetection> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PS C:\Users\chong\Downloads\Group05_FruitAndVegetablesDetection> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\chong\Downloads\Group05_FruitAndVegetablesDetection>
```

Step 3

1. Enter “pip install -r 05_Installation_and_User_Guide/requirements.txt”

The requirement source will be download



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like '02_Source_Code', '04_Trained_Model', '05_Installation_and_User_Guide', and 'venv'. The code editor displays a README file with instructions for installation and usage. A red arrow points from the first instruction box to the terminal window, which shows the command 'pip install -r 05_Installation_and_User_Guide/requirements.txt' being executed. The terminal output shows the installation progress for various packages like streamlit, plotly, torch, torchvision, ultralytics, pillow, numpy, and torchmetrics.

```
05_Installation_and_User_Guide > ① README.md > abc # Insurance Scam Detection — Model Comparison UI > abc ## Installation
1  # Insurance Scam Detection – Model Comparison UI
7  ## Folder structure expected
21 Place "trained_model/" in the same directory as "app.py" (or update the
22 "MODEL_DIR" / "*"PATH" variables at the top of "app.py" to point elsewhere).
23
24 ## Installation
25
26 ```bash
27 python -m venv venv
28 ## Mac
29 source venv/bin/activate      # Windows: venv\Scripts\activate
30
31 ## If using Visual Studio Code, use below command to enter venv stage
32 .\venv\Scripts\Activate.ps1
33
34 pip install -r 05_Installation_and_User_Guide/requirements.txt
35
36
37 Requires Python 3.9+ and, ideally, a CUDA-capable GPU (the app falls back
38 to CPU automatically if none is found).
39
40 ## Running the app
41
42 ```bash
43 streamlit run 02_Source_Code/app.py
44
45
46 This opens the UI in your browser (default: http://localhost:8501).
47
48 ## Usage
```

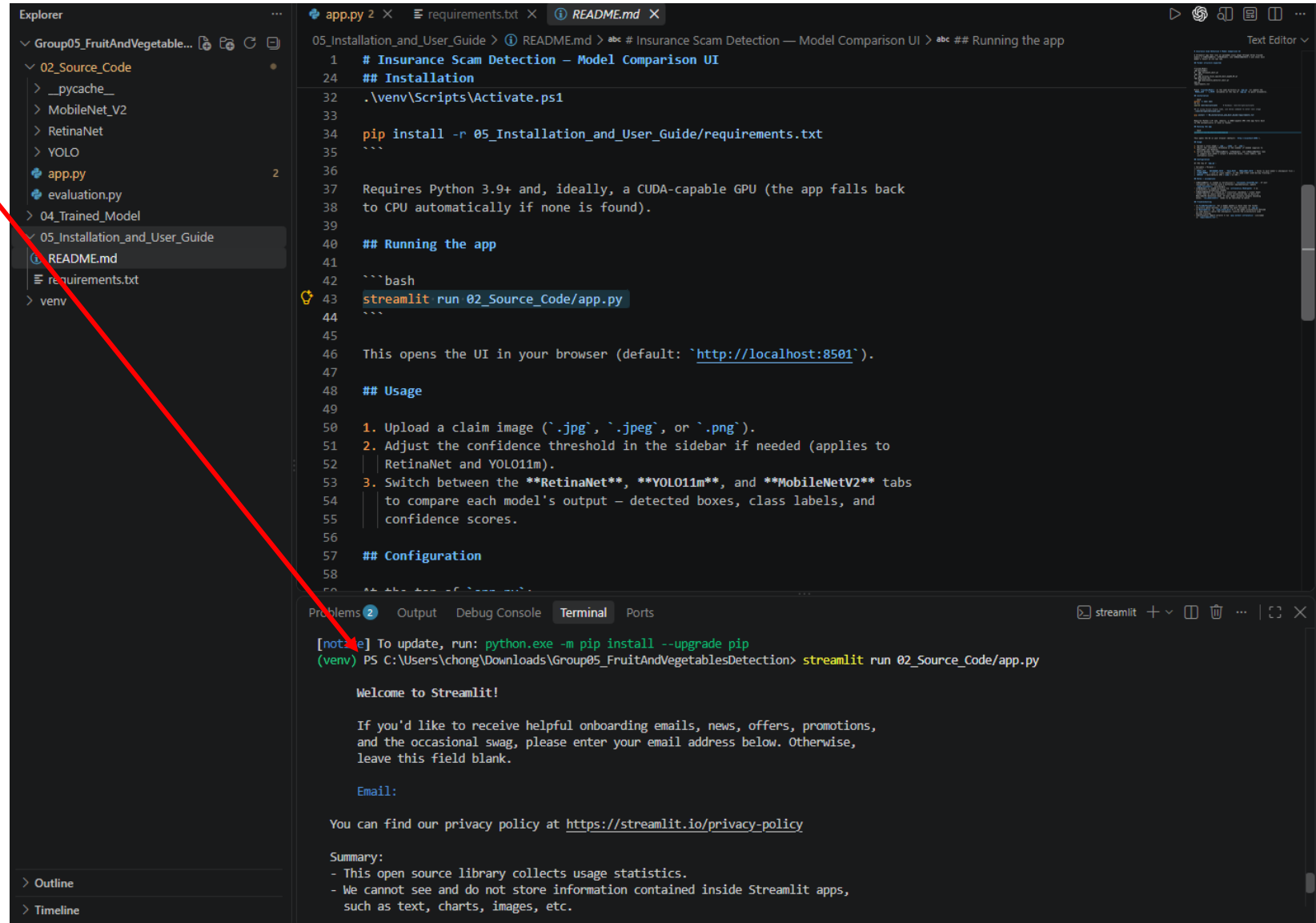
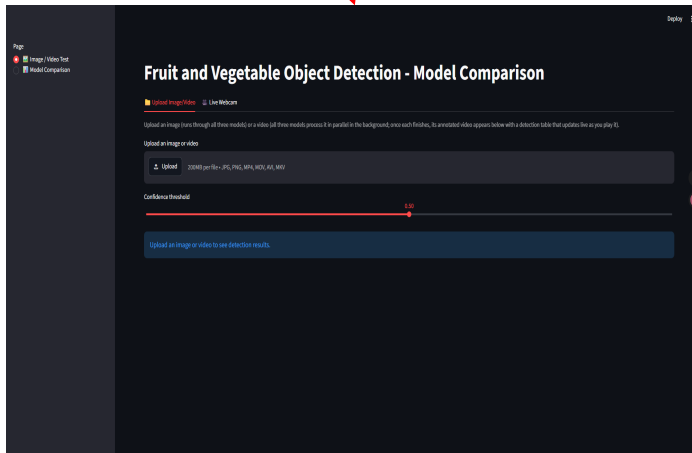
Problems Output Debug Console Terminal Ports

```
(venv) PS C:\Users\chong\Downloads\Group05_FruitAndVegetablesDetection> pip install -r 05_Installation_and_User_Guide/requirements.txt
Collecting streamlit (from -r 05_Installation_and_User_Guide/requirements.txt (line 1))
  Using cached streamlit-1.62.0-py3-none-any.whl.metadata (10 kB)
Collecting plotly (from -r 05_Installation_and_User_Guide/requirements.txt (line 2))
  Using cached plotly-7.0.0-py3-none-any.whl.metadata (8.7 kB)
Collecting torch (from -r 05_Installation_and_User_Guide/requirements.txt (line 3))
  Using cached torch-2.13.0-cp311-win_amd64.whl.metadata (39 kB)
Collecting torchvision (from -r 05_Installation_and_User_Guide/requirements.txt (line 4))
  Using cached torchvision-0.28.0-cp311-win_amd64.whl.metadata (5.6 kB)
Collecting ultralytics (from -r 05_Installation_and_User_Guide/requirements.txt (line 5))
  Using cached ultralytics-8.4.130-py3-none-any.whl.metadata (45 kB)
Collecting pillow (from -r 05_Installation_and_User_Guide/requirements.txt (line 6))
  Using cached pillow-12.3.0-cp311-win_amd64.whl.metadata (9.3 kB)
Collecting numpy (from -r 05_Installation_and_User_Guide/requirements.txt (line 7))
  Using cached numpy-2.4.6-cp311-win_amd64.whl.metadata (6.6 kB)
Collecting torchmetrics (from -r 05_Installation_and_User_Guide/requirements.txt (line 8))
  Using cached torchmetrics-1.9.0-py3-none-any.whl.metadata (23 kB)
```

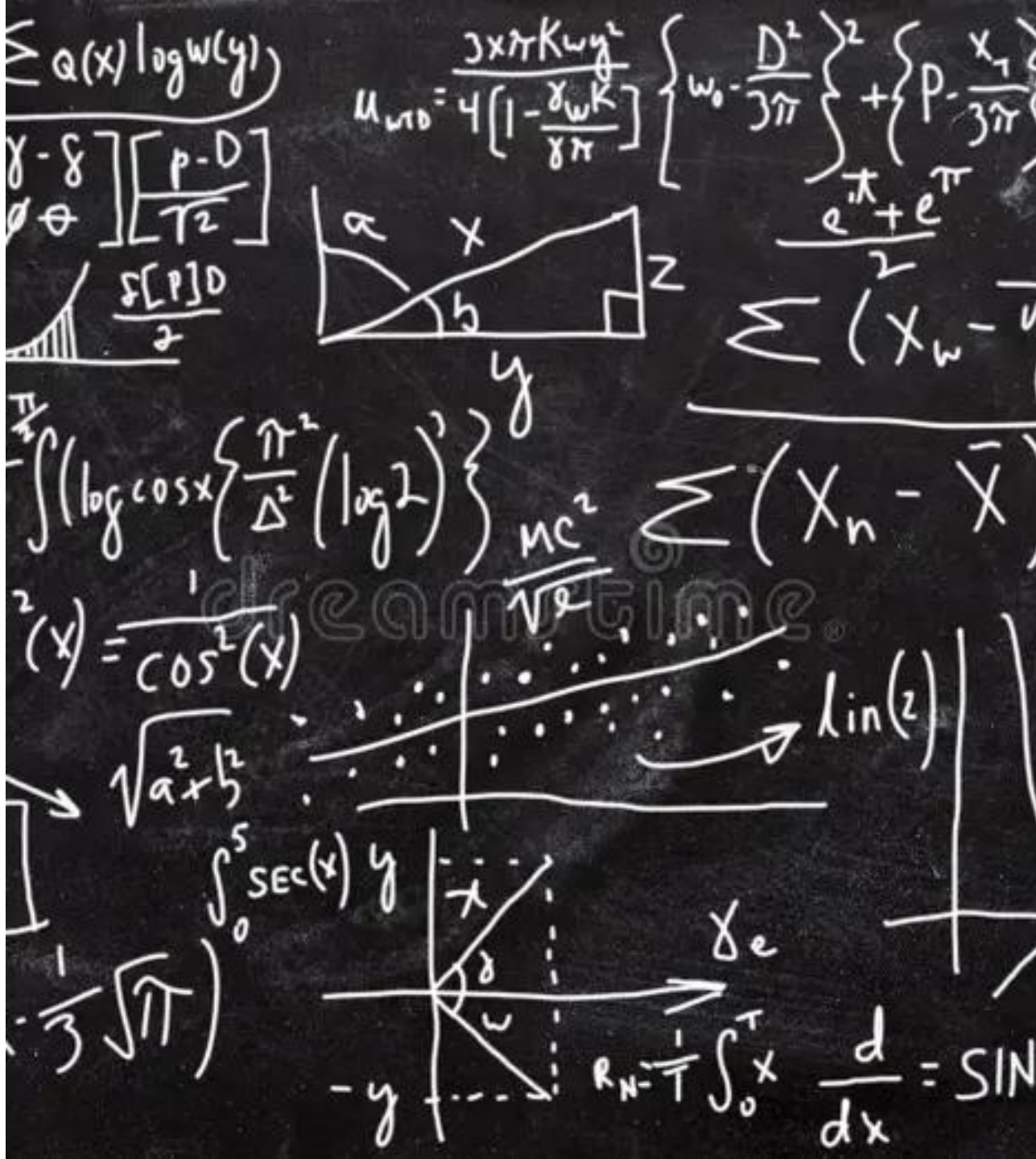

Step 4

1. Enter “streamlit run 02_Source_Code/app.py”

It will open a webpage



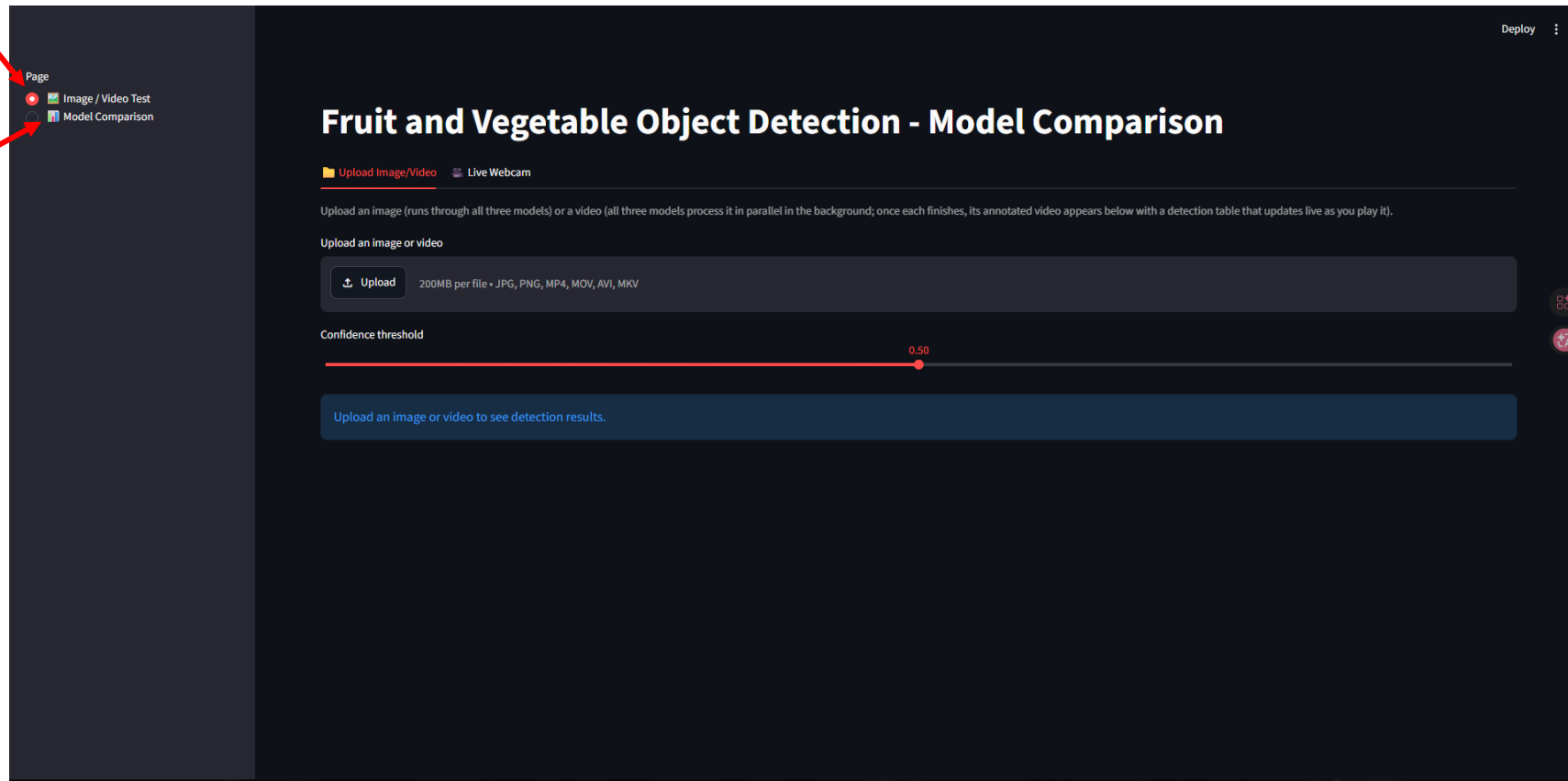
Stream lit



Stream Lit Application

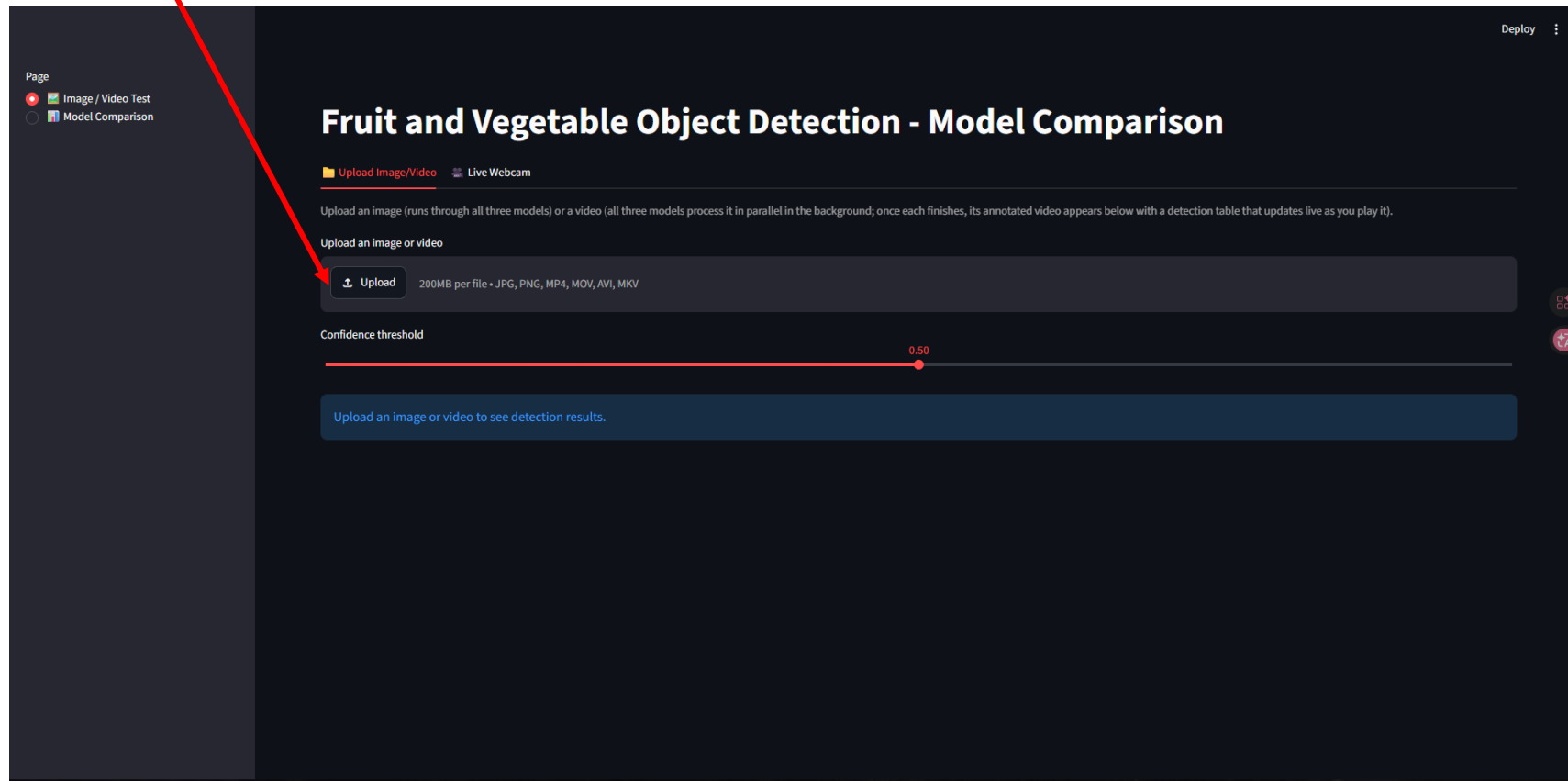
Main page

Model
trained
performance



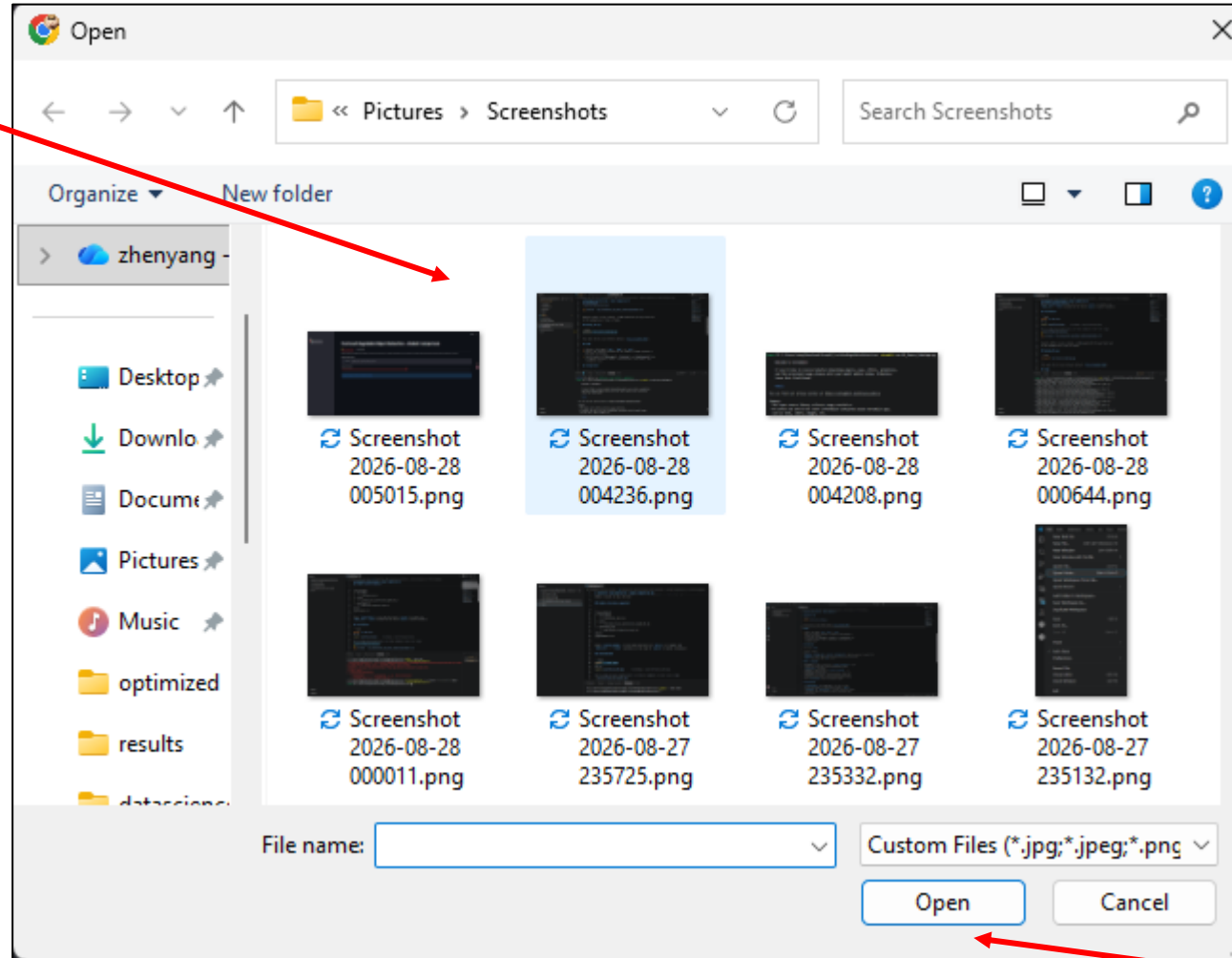
Upload Image (Step 1)

1. Click “upload”



Upload Image (Step 2)

1. Select picture

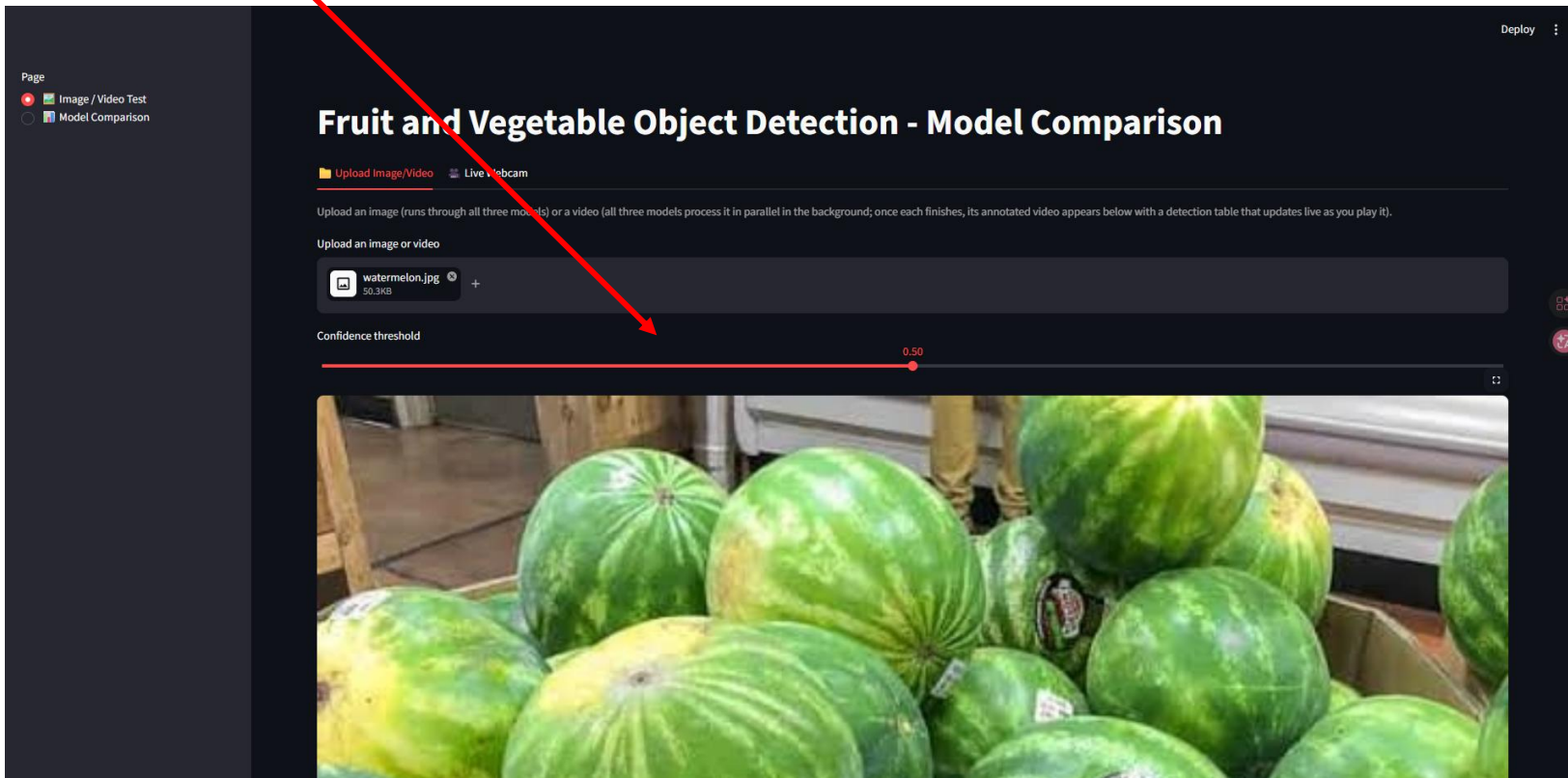


2. Click "Open" to confirm

Upload Image (Step 3)

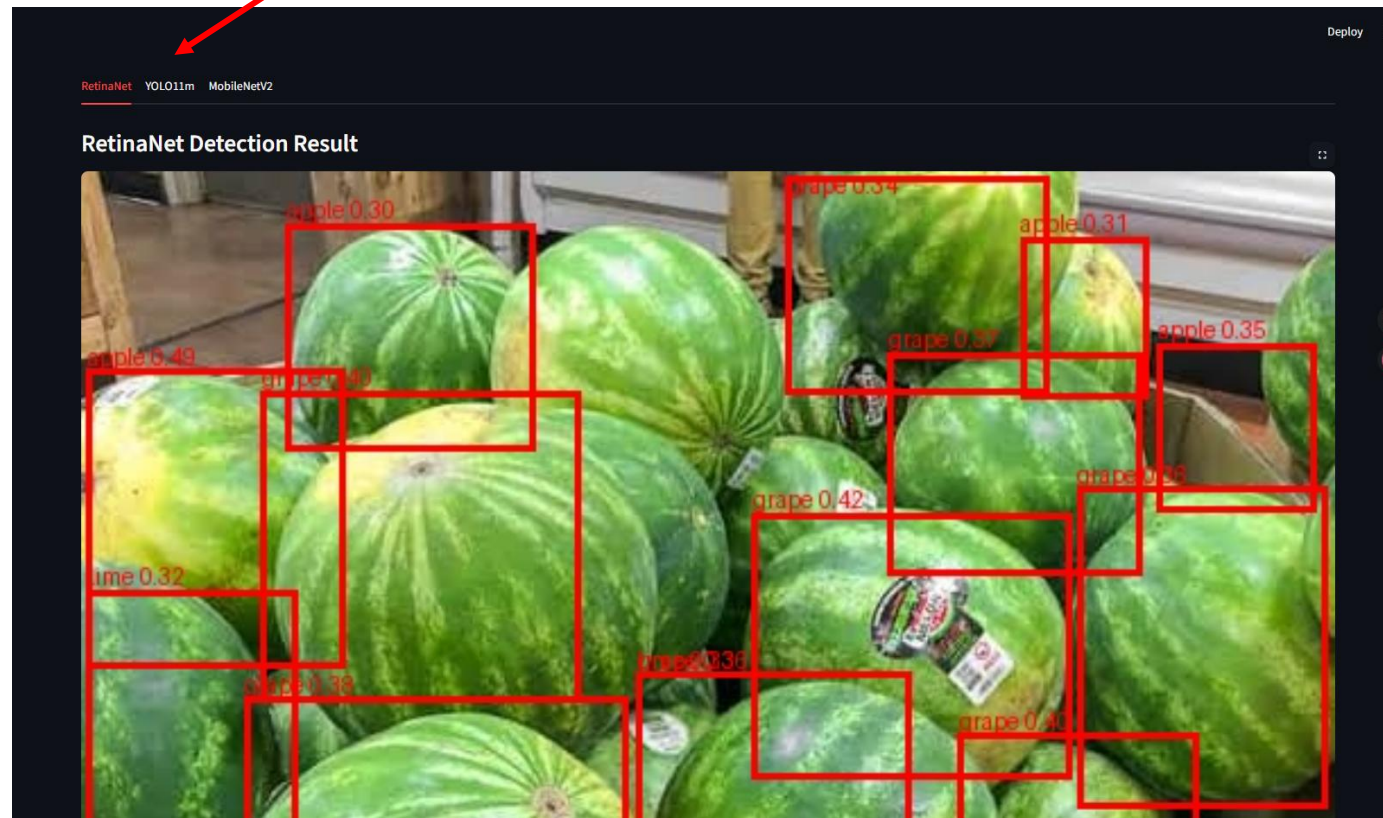
1. Drag the Confidence Threshold

The confidence threshold represents the minimum confidence score required for a detected object to be displayed.



Upload Image (Step 4)

2. Select the different model by click on the title of these models



Upload Image (Step 5)

Inference time is the time used to the image

class: object class

confidence: accuracy

box: the bounding area in the picture

Inference time: 0.426s

class	confidence	box
apple	0.4912	[2.7,87.9,116.1,219.2]
grape	0.4153	[296.8,151.2,437.6,268]
grape	0.4016	[387.8,249,493,359.9]
grape	0.3955	[79.7,97.4,220.5,234.3]
grape	0.3753	[72,232.7,241.7,391]
grape	0.3749	[357,80.1,468.1,178.8]
lime	0.3734	[10.4,342.8,81.3,385.9]
grape	0.3615	[245.6,221.8,366.3,373.4]
grape	0.3581	[440.2,139.9,550.9,281.1]
apple	0.3465	[475.8,76.3,545.4,150.6]
grape	0.3405	[311.7,2.1,427.9,98.2]
lime	0.3277	[245.6,221.8,366.3,373.4]
lime	0.3228	[2.5,185.4,95.7,354.6]
apple	0.3068	[415.2,29.4,471.8,101]
apple	0.3020	[90.7,23.1,200.1,123.8]

Live Webcam

1. Select model

4. Click "START"

Deploy

Fruit and Vegetable Object Detection - Model Comparison

Upload Image/Video **Live Webcam**

Runs inference on your webcam feed in real time, with a live-updating detection table beside it. YOLO11m is the fastest and most likely to keep up with live video; RetinaNet and MobileNetV2 may lag on slower GPUs/CPU's since every frame is a full forward pass.

Model: YOLO11m

Confidence threshold: 0.50

Change settings above, then click Start again to apply them to a new session. To run detection on a screen recording instead of the camera, record your screen with your OS's screen recorder and upload it in the 'Upload Image/Video' tab - the same parallel processing and live table apply there.

Live detections

Click Start to begin.

START

SELECT DEVICE

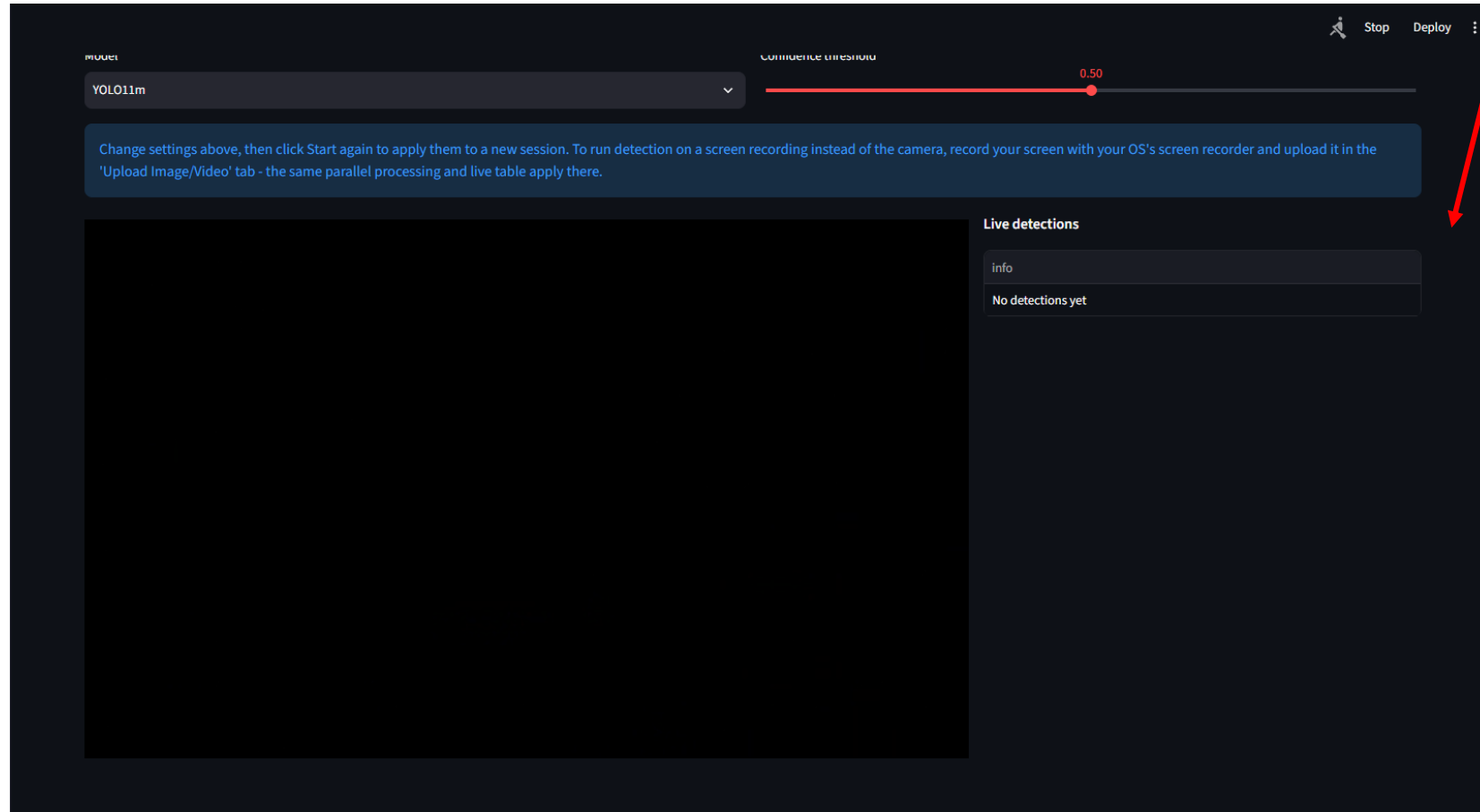
2. Select confidence threshold

3. Select device

Please allow the camera permission to this website when using this function

Live Webcam (2)

The detection result will show at this side



Model Performance

1. Click “Model Comparison” in left panel

In this page, you can view all the model performance

